Building Frontend Web Apps with React

*Co-teachers: **Vitor** and **Marga***

Thu. 2.4.26

Outline

01 Warm-Up

Check in & "Status Quo" 🤓

15 min

02 Setup

React project from scratch, rendering

10 min

03 React - props

Passing ***unchangeable*** values from Parent to Child

30 min

04 Break

10 min

05 React - state mgmt

25 min

useState for values that ***change***

06 React - fetch API

20 min

Map results on the page

01 Warm-Up

Check in & Status Quo - where do we stand?

02 Setup

Starting React project from scratch

```
npm create vite@latest
```

Select: **React** , **JavaScript** , let **npm install** run and start.

In folder (parent-dir) always: `npm install & npm run dev`

README: 01_ecommerce/02_milestone

03 React - props

Passing ***unchangeable*** values from Parent to Child - via the props object

```
When React renders `<Greeting name="Alice" />`, it **automatically
creates** this object and passes it as the first parameter:

```jsx
function Greeting(props) {
 // props = { name: "Alice" }
 //           ↑key     ↑value (immutable!)
}
```

So `props.name` accesses the **value** `"Alice"` from the key-value
pair `name: "Alice"`. The child **cannot change** `props.name`.
```

React props - w3schools

README: 01_ecommerce/02_milestone

03 React - props

Passing ***unchangeable*** values from Parent to Child

```
// 1. Via the props object (more verbose)
function Greeting(props) {
 console.log(props); // { name: "Alice" }
 return <h1>{props.name}</h1>;
}
```

- via the props object ...

```
// 2. Directly via destructuring (cleaner)
function Greeting({ name }) {
 console.log(name); // Alice
 return <h1>{name}</h1>;
}
```

- via destructuring

React props - w3schools

README: 01_ecommerce/02_milestone

03 React - props

```
function Greeting(props) {
 return (
   <div>
     <h1>Hello, {props.name}!</h1>        // Access value by key
     <p>Age: {props.age}</p>              // Access another value
     <p>From: {props.city}</p>
   </div>
 );
}
```

Destructure

take specifically those values you want to have (e.g. to display)

```
function Greeting({ name, age, city }) {  // React extracts values automatically
 return (
   <div>
     <h1>Hello, {name}!</h1>     // Direct access to values
     <p>Age: {age}</p>
     <p>From: {city}</p>
   </div>
 );
}
```

README: 01_ecommerce/02_milestone

03 React - props

Issue

Missing props => **undefined**

```
// in App.jsx:
<Greeting name="Alice" /> // ← only name passed


// in Greetings.jsx component:
function Greeting(props) {
 console.log(props);
 // Output: { name: "Alice" }
  return (
   <div>
     <h1>Hello, {props.name}!</h1>      // "Alice" ✅
     <p>Age: {props.age}</p>            // "Age: undefined" ❌
     <p>From: {props.city}</p>          // "From: undefined" ❌
   </div>
 );
}
```

Output in browser:

Hello, Alice!
Age:
From:

README: 01_ecommerce/02_milestone

03 React - props

Solutions

(*Problem: Missing props => undefined*)

```
function Greeting(props) {
 return (
   <div>
     <h1>Hello, {props.name}!</h1>
     {props.age && <p>Age: {props.age}</p>}      // Only if truthy
     {props.city && <p>From: {props.city}</p>}
   </div>
 );
}
```

Fix with

conditional rendering

Problem: Pass only `name` → show `**undefined**` blanks

Solution 1. Add `**{props.age && ...}**` → blanks disappear

Solution 2. Show **destructuring defaults** → clean output

README: 01_ecommerce/02_milestone

03 React - props

Solutions

(*Problem: Missing props => undefined*)

```
function Greeting({ name, age = 30, city = "Unknown" }) {
 return (
   <div>
     <h1>Hello, {name}!</h1>
     <p>Age: {age}</p>
     <p>From: {city}</p>
   </div>
 );
}
```

Fix with

default values

Problem: Pass only `name` → show `**undefined**` blanks

Solution 1. Add `**{props.age && ...}**` → blanks disappear

Solution 2. Show **destructuring defaults** → clean output

Break!

See you again in
10 Minutes 😉

surprise inside

05 React State Management

useState for managing values that **change**

So far, our components could receive and display values through **props**.

But once values need to change we need **state**.

— for example a favorite button, a counter, or fetched products —

05 React hooks

Props

- passed from Parent to Child
- used to read and display date
- do not change in the child

(props are **immutable**)

State

- lives in the component
- can change over time
- updates the screen when the

value changes

React becomes dynamic when changing data is rendered again!

05 React hooks

- useState
- useEffect

  => Fetching API

  (***persistance*** *is relevant; localStorage, JSON and later: MongoDB*)

Hooks is the broader React concept.

Today we only need useState **to manage changing data** and useEffect **to fetch products.**

surprise inside

Final questions?

❗ let’s make a group picture 😁